Contenedores y GitHub Actions en IMSI

Documento de decisión. Responde tres preguntas concretas —¿caben contenedores?, ¿se recomiendan? y ¿cómo encajan los Actions?— y justifica en cada caso **por qué sí** y **por qué no**, porque ninguna de las dos herramientas es buena o mala en abstracto: dependen del tamaño del equipo, del servidor y de lo que ya exige el análisis.

El proyecto es un **monolito modular con arquitectura hexagonal**: un proceso Django + DRF, workers Celery, PostgreSQL, nginx, frontend Next.js y un disco externo para respaldos. Los requisitos que tocan este documento son NFR-04-02 (transacciones), NFR-05-02 (pipeline, cobertura ≥ 70 %, datos sintéticos, bloqueo de despliegue) y NFR-07-01 (escalabilidad del asistente).

---

1. La respuesta corta

Pregunta	Respuesta	Por qué
¿Caben contenedores?	**Sí**, sin tocar la arquitectura	Docker empaqueta; hexagonal organiza dependencias. Son dimensiones distintas
¿Se recomiendan?	**Sí, con Docker Compose** y un solo host	Equipo de cinco personas, un servidor en red local, sin necesidad de orquestador
¿Y Kubernetes?	**No**	Añade operación que nadie del equipo va a mantener; el sistema no lo necesita
¿GitHub Actions?	**Sí, es obligatorio**	NFR-05-02 lo exige explícitamente, con cobertura y bloqueo
¿Actions evita conflictos de merge?	**No**	Detecta errores después del merge; los conflictos se evitan con ramas cortas y rebase
¿Se pueden escribir los workflows con IA?	**Sí, con revisión**	Es YAML; el riesgo es aceptar un pipeline que no bloquea nada

---

2. Contenedores: qué resuelven y qué no

2.1 Qué es un contenedor (y qué no)

Un contenedor empaqueta una aplicación con sus dependencias en una imagen ejecutable. **No** es una máquina virtual (comparte el kernel), **no** es un microservicio y **no** cambia la arquitectura del código: el mismo monolito puede correr en un contenedor o en un `venv`.

Consecuencia práctica: el diagrama hexagonal no se modifica. Lo que cambia es dónde corre cada proceso y cómo se despliega.

2.2 Cuándo sí: los casos donde gana

Caso	Por qué sí
Cinco personas con Windows, macOS y Linux	La misma imagen corre igual en las tres; desaparece el «en mi máquina funciona»

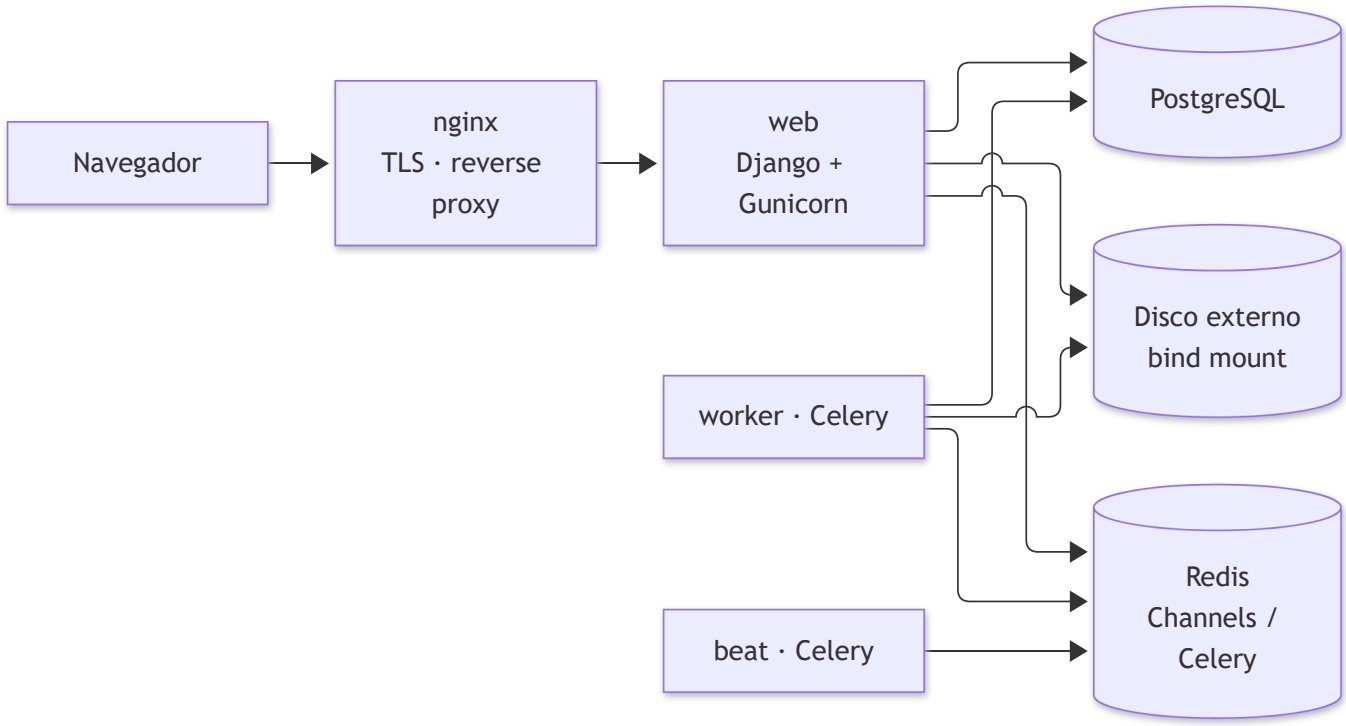
Caso	Por qué sí
CI que construye lo mismo que se despliega	Actions construye la imagen y esa imagen es la que va al servidor; no hay dos formas de instalar
Servidor propio en la red local	`docker compose` levanta nginx, Django, workers, beat y PostgreSQL con un comando
Volver atrás rápido	Se despliega por etiqueta de imagen; revertir es usar la anterior
Aislar versiones	PostgreSQL 16 y Python 3.12 fijos, sin tocar el sistema operativo del servidor
Reproducibilidad	Dependencias fijadas por `requirements.txt` y digest de imagen

2.3 Cuándo no: los casos donde estorba

Caso	Por qué no
Un solo desarrollador, sin servidor	El coste de mantener Dockerfile y compose supera el beneficio de reproducibilidad
Equipo sin experiencia en Docker	Un `volume` mal montado o un `depends_on` sin healthcheck producen fallos silenciosos peores que un `venv`
Disco USB de respaldos como dispositivo	En Windows el contenedor no ve el dispositivo USB crudo; hay que montar una ruta del host o usar WSL2 con passthrough. Si el respaldo depende de eso, el montaje debe probarse antes de prometerlo
Base de datos pequeña y local	Un PostgreSQL instalado en el host es más simple de respaldar y de depurar
Necesidad de GUI o drivers	No aplica aquí, pero es el motivo típico por el que un contenedor no sirve

2.4 La decisión para IMSI

Docker Compose, un solo host, cinco servicios. No Kubernetes, no Swarm, no servicios separados por módulo.



Justificación de cada pieza:

- **Un Dockerfile, tres procesos** (web, worker, beat). Misma imagen, distinto comando. Evita que la web y el worker tengan dependencias distintas.
- **PostgreSQL con volumen nombrado**, no con datos dentro del contenedor.
- **Redis** solo si se usa Channels con capa de canales o una cola externa; si todo va en memoria, sobra.
- **nginx** como único punto expuesto; Django no publica puerto.
- **El disco externo con `bind`**, nunca con volumen anónimo: si el montaje se pierde, el respaldo se escribe dentro del contenedor y desaparece al recrearlo.

2.5 Lo que hay que cuidar (y cómo)

1. **Respaldo del volumen no es respaldo.** El volumen de PostgreSQL vive en el host; el respaldo real es el `pg_dump` cifrado que va al disco externo, con su prueba de restauración (CU-006).
2. **Secretos fuera de la imagen.** Variables de entorno o `docker secret`; nunca en el `Dockerfile` ni en el repositorio.
3. **Healthchecks reales.** `depends_on: condition: service_healthy` en base y web; sin eso, los contenedores arrancan en desorden.
4. **Usuario no root** en la imagen y sin `--privileged`.
5. **Etiquetas por versión y digest**, no `latest` en producción.
6. **Logs a la salida estándar**, para que `docker logs` y el sistema de la empresa los vean.

2.6 Comparación con las alternativas

Criterio	`venv` en el host	Docker Compose	Kubernetes	PaaS
Reproducibilidad	baja	alta	alta	alta
Operación que hay que mantener	baja	media	alta	baja

Criterio	`venv` en el host	Docker Compose	Kubernetes	PaaS
Encaja con un servidor local	sí	sí	no (sobra)	no (sale de la red)
Coste para cinco estudiantes	bajo	medio	alto	medio
Rollback	manual	por etiqueta	por despliegue	por despliegue
Adecuado para IMSI	aceptable	**sí**	no	no

---

3. GitHub Actions: qué resuelve y qué no

3.1 Lo que el requisito exige

NFR-05-02 no deja margen: pipeline en GitHub Actions, compilación, pruebas unitarias e integrales **con datos sintéticos**, 100 % de pruebas en verde y cobertura mínima del 70 %, con bloqueo del despliegue si falla. Es un requisito del análisis, no una mejora opcional.

3.2 Qué sí resuelve

Problema	Cómo lo resuelve
Codigo que rompe `main`	Los checks fallan antes de fusionar (con protección de rama)
Cobertura que baja poco a poco	`--cov-fail-under=70` la frena en seco
Modelo cambiado sin migración	`makemigrations --check --dry-run`
El dominio importa Django	`import-linter` verifica la regla del capítulo 3
Imagen que no construye	`docker build` en cada PR
Estilo y bugs obvios	`ruff`
Dependencias con vulnerabilidades	`pip-audit` (opcional, recomendado)

3.3 Qué **no** resuelve

Expectativa equivocada	Realidad
«Con Actions no hay conflictos de merge»	Un conflicto lo produce tocar las mismas líneas. Actions corre **después**; no evita el conflicto
«Si pasa CI, está bien»	Con 70 % de cobertura, un tercio del código no está protegido
«CI reemplaza la revisión»	Un PR sin revisión sigue siendo un PR sin revisión
«CI valida las migraciones»	`--check` detecta migraciones faltantes, no que la migración destruya datos
«CI prueba con datos reales»	NFR-05-02 lo prohíbe: datos sintéticos

3.4 El pipeline recomendado

```yaml
name: ci
on: [push, pull_request]
jobs:
```

```yaml
calidad:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16
      env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: imsi_test }
      ports: ["5432:5432"]
      options: >-
        --health-cmd pg_isready --health-interval 10s
        --health-timeout 5s --health-retries 5
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.12" }
    - run: pip install -r requirements.txt
    - run: ruff check .
    - run: python manage.py makemigrations --check --dry-run
    - run: pytest --cov --cov-fail-under=70
    - run: lint-imports
    - run: docker build -t imsi:${{ github.sha }} .
```

Y fuera del YAML, en la configuración del repositorio:

1. **Protección de `main`**: exigir PR, exigir checks en verde, exigir una revisión, prohibir push directo.
2. **Ramas cortas**: una rama por tarea, integrada en el mismo día.
3. **Rebase antes del PR** para que el merge sea lineal y el conflicto aparezca temprano, no al final del semestre.

3.5 Cuándo un CI mínimo es suficiente

Para un proyecto de un mes con un solo desarrollador, `ruff` + `pytest` + build alcanza. Todo lo demás se justifica cuando hay **más de una persona** o cuando el requisito lo exige, que es este caso.

---

4. ¿Se pueden escribir los workflows con un asistente de IA?

Sí. Un workflow es YAML, y un asistente lo escribe, lo corrige y lo explica igual que cualquier archivo de texto. La pregunta correcta no es si se puede, sino **qué hay que revisar antes de confiar en él**.

4.1 Dónde ayuda

- Generar el esqueleto del pipeline y la matriz de versiones.
- Traducir un requisito («cobertura mínima de 70 %») a la bandera correcta.
- Añadir caché de dependencias y servicios de PostgreSQL.
- Explicar por qué falla un job.

4.2 Dónde hace daño si no se revisa

Riesgo	Ejemplo	Cómo evitarlo
Pipeline que no bloquea	Un `|| true` al final del `pytest`	Leer el YAML línea por línea
Acción de terceros sin fijar	`uses: alguien/accion@main`	Fijar por versión o por SHA
Inyección por el título del PR	`run: echo "${{ github.event.pull_request.title }}"`	Usar variables de entorno, nunca interpolar directamente
`pull_request_target` con secretos	Workflow que ejecuta código del fork con permisos del repo	Evitarlo o no exponer secretos
Secretos en logs	`echo $TOKEN` para depurar	Nunca imprimir; usar `::add-mask::`
Pruebas con datos reales	Cadena de conexión de producción en CI	Datos sintéticos (NFR-05-02)

4.3 El criterio

Se puede pedir el pipeline a un asistente; lo que no se puede es **aceptarlo sin leerlo**. La prueba es simple: si el YAML contiene `|| true`, `continue-on-error: true` en un paso crítico, o una acción sin versión, no es un pipeline: es un adorno.

---

5. Decisión por escenario

Escenario	Contenedores	Actions	Comentario
Curso, cinco personas, servidor de la empresa	Compose en un host	Pipeline completo + protección de rama	Es el caso de IMSI: los dos valen y están justificados
Un desarrollador, demo local	`venv` basta	`ruff` + `pytest`	Docker añade más de lo que resuelve
Crecer a varios servidores	Añadir balanceador y réplicas de lectura	Añadir despliegue por etiqueta	No pasar a Kubernetes sin operador dedicado
Requisito de alta disponibilidad 99,9 %	Compose se queda corto	Necesita despliegue orquestado	Ese requisito no existe en el análisis actual

---

6. Anexo: compose y Dockerfile de referencia

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: imsi
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: [ pgdata:/var/lib/postgresql/data ]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5
```

```yaml
  web:
    build: .
    command: gunicorn config.wsgi:application --bind 0.0.0.0:8000
    env_file: .env
    depends_on:
      db: { condition: service_healthy }
    volumes: [ "/mnt/respaldos:/respaldos" ]

  worker:
    build: .
    command: celery -A config worker -l info
    env_file: .env
    depends_on: [ db, web ]

  beat:
    build: .
    command: celery -A config beat -l info
    env_file: .env
    depends_on: [ db ]

  nginx:
    image: nginx:1.27
    ports: ["443:443"]
    volumes: [ "./nginx.conf:/etc/nginx/nginx.conf:ro" ]
    depends_on: [ web ]

volumes:
  pgdata:
```

```dockerfile
FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
RUN adduser --disabled-password --gecos "" appuser
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER appuser
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

---

7. Cierre

- Los contenedores **no cambian la arquitectura**: hexagonal sigue igual; cambia el empaquetado y el despliegue.
- Compose es la herramienta correcta para un monolito modular en un servidor local; Kubernetes es sobrecoste sin operador.
- Actions es obligatorio por NFR-05-02, y su valor real está en la protección de rama y en las pruebas que bloquean, no en el archivo YAML.

- Los Actions **no evitan conflictos**: los detectan. Contra los conflictos, ramas cortas y rebase.
- Un asistente puede escribir el pipeline; la revisión humana es la que decide si ese pipeline sirve.